

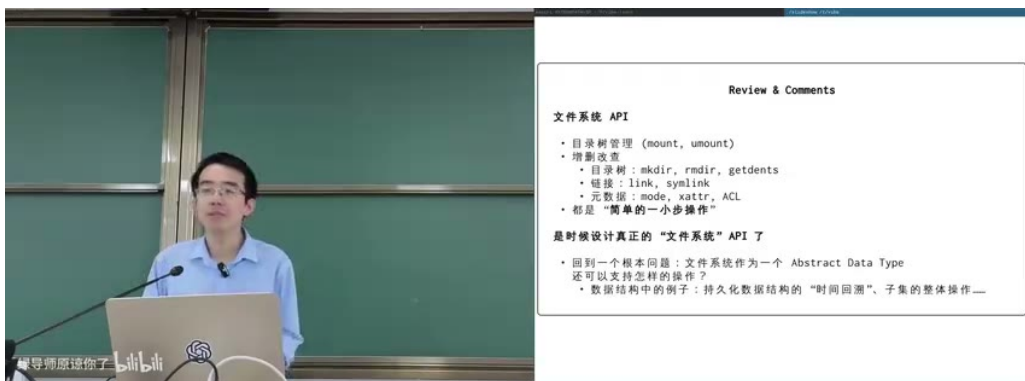
课程笔记

25 - 文件系统 API (2)

2026 南京大学操作系统原理

lecture-to-notes

2026-06-20



视频作者/频道: 南京大学操作系统原理 / 蒋炎岩

发布日期: 2026

视频时长: 01:22:38

视频链接: 本地课程视频

目录

1	从 CRUD 到文件系统级 API	3
1.1	文件系统是一个抽象数据类型	3
1.2	四类高级需求	3
1.3	本章小结	4
2	文件系统监控：从轮询到事件	4
2.1	为什么需要监控	4
2.2	inotify：把变化变成文件描述符上的事件	5
2.3	watchdog 示例：事件比想象更细	6
2.4	本章小结	7
3	从文件监控到操作系统状态监控	7
3.1	trace 是状态机执行的信息投影	7
3.2	eBPF：把受限代码放到内核 hook point	7
3.3	Specification is all you need	8
3.4	本章小结	9
4	快照：持久化数据结构与 Git	9
4.1	append-only + random read	9
4.2	Git 对象：blob、tree、commit	10
4.3	Btrfs：把整个文件系统做成持久化数据结构	12
4.4	本章小结	12
5	OverlayFS 与 Docker：目录树的叠加语义	12
5.1	UnionFS/OverlayFS 的新语义	12
5.2	应用：试升级、考试环境、网吧管理	13
5.3	Docker layer：每个 RUN 都是一层	14
5.4	本章小结	14
6	FUSE：把任意数据结构伪装成文件系统	15
6.1	为什么不能自己实现文件系统	15
6.2	GGFS：文件系统驱动 Galgame	15
6.3	FUSE hacks：把非文件变成文件	16
6.4	本章小结	17
7	总结与延伸	17
7.1	课程结论	17
7.2	概念压缩	18
7.3	面向 AI 时代的延伸	18

7.4 本章小结	18
--------------------	----

1 从 CRUD 到文件系统级 API

1.1 文件系统是一个抽象数据类型

上一讲讨论的是文件和目录的基本增删改查：`mkdir`、`rmdir`、`getdents`、`link`、`unlink`、`symlink`、`read`、`write`、权限位、扩展属性等。它们都很像“对一个节点做一次局部操作”。这一讲把视角提升到整个文件系统：如果文件系统是一棵目录树、一组对象、一种抽象数据类型，那么我们不只可以定义点状 CRUD，还可以定义整棵树上的高级操作。

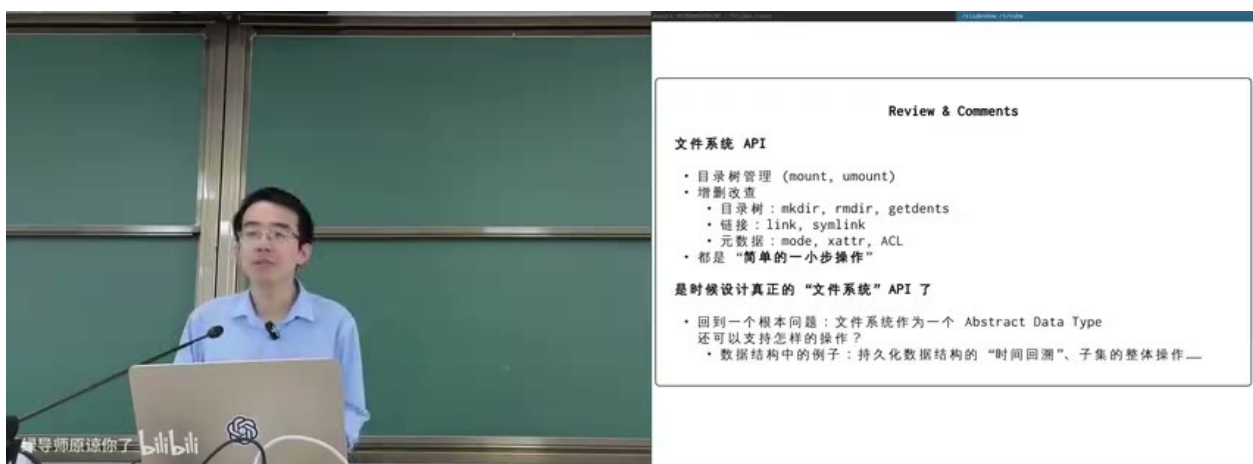


图 1: 课程开头回顾上一讲的目录树管理、增删改查和挂载，并把问题提升为：能否设计真正的文件系统级 API？¹

本讲的统一视角

文件系统不是一堆系统调用的清单，而是一个数据结构。既然它是数据结构，就可以讨论监控、快照、覆盖、虚拟化、用户态实现等“结构级”操作。今天的主线是：从一个目录里的文件变化，逐步扩展到整个操作系统状态、持久化目录树、容器镜像层，以及任意数据结构伪装成文件系统。

课程一开始用数据结构课里的例子做动机：如果维护一个整数集合，给所有元素加 10 不一定要真的遍历所有元素；也可以把一个全局偏移量记录在数据结构旁边，查询时再解释。如果插入逻辑也一起调整，这个 trick 就能改变操作的复杂度。同理，目录树也可以有高级解释层：读到的文件可能来自底层目录、上层目录、历史快照、远程机器，甚至来自一段程序即时生成的结果。

1.2 四类高级需求

这一讲实际围绕四类需求展开：

¹视频画面时间区间：00:00:00–00:00:05；附近讲解：复习文件系统 API，指出上节课主要是“简单的一小步操作”，本讲讨论文件系统作为抽象数据类型的更高级操作。

- **监控**：文件系统改变后通知应用，避免反复扫描全树。
- **快照**：把某一时刻的目录树根指针留下来，之后仍能回到旧版本。
- **覆盖**：把多个目录层叠成一个虚拟目录，为容器、试升级和多用户隔离服务。
- **虚拟化**：自己实现一个文件系统，让任意数据结构或服务以目录树形式暴露。

这些需求共享一个核心事实：`mount` 把文件系统接入统一目录树，文件描述符把对象接入统一 I/O 模型，而系统调用给用户态一个受控入口。只要能在这些入口处改变解释规则，文件系统就不再只是磁盘上的字节，而是操作系统中的可编程对象。

1.3 本章小结

文件系统 API (2) 的重点不是再背一批系统调用，而是理解“目录树作为数据结构”。从这个角度看，监控是增量事件，快照是持久化数据结构，覆盖是目录树合成，FUSE 是用户态实现这个抽象数据类型。

2 文件系统监控：从轮询到事件

2.1 为什么需要监控

文件变化触发动作是现实软件的日常需求。Web 开发框架在 debug mode 下监控源码变化并重启服务；前端工具链保存 CSS 后即时刷新浏览器；文件管理器需要自动显示终端里创建的文件；编辑器要发现磁盘文件被外部程序改写；`tail -f` 要等待日志继续增长。这些都要求系统告诉应用：哪个文件发生了什么变化。

最朴素的办法是轮询。对目录树中的所有文件执行 `stat`，记录 `mtime`，等待一段时间后再执行一次，然后 `diff` 两份结果。



图 2: 轮询式监控：用 `stat` 记录全目录树的修改时间，再用 `diff` 判断哪些文件变化。²

²视频画面时间区间：00:04:05–00:04:10；附近讲解：用 `stat -c`、shell 进程替换和 `diff` 可以从 CRUD 操作中拼出文件变化检测，但目录很大时复杂度不理想。

如果目录树中有 N 个文件，一次轮询需要访问 N 个元数据项；如果真实变化只有 k 个文件，事件式机制只需要处理 k 条事件。可以粗略写成：

$$T_{\text{poll}} = O(N), \quad T_{\text{event}} = O(k)$$

符号说明：

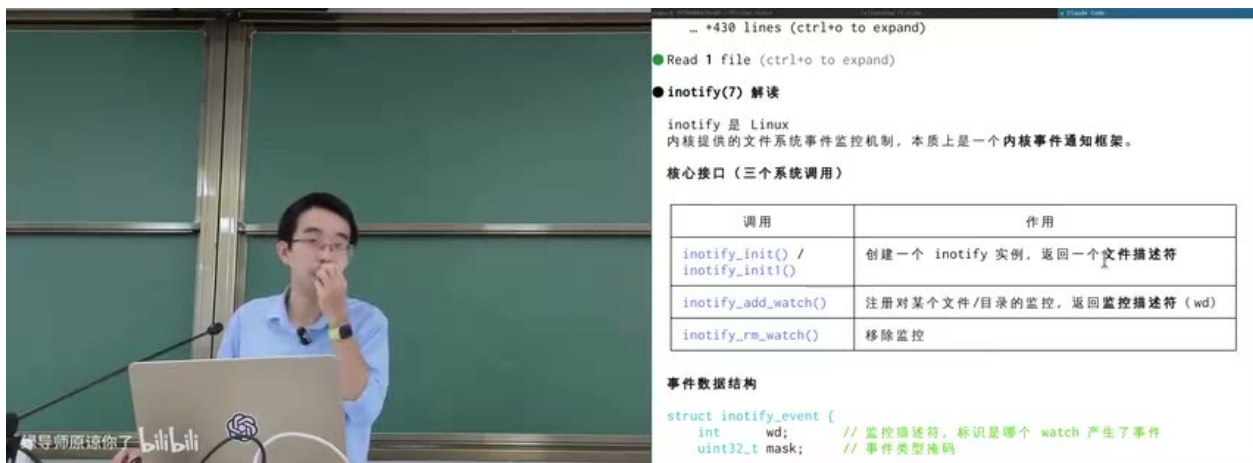
- N ：被监控目录树中的文件和目录数量；
- k ：一次时间窗口内真正发生变化的对象数量；
- T_{poll} ：轮询扫描带来的元数据查询成本；
- T_{event} ：内核按变化产生事件后，用户态消费事件的成本。

轮询正确但不优雅

轮询不是错的。它完全基于已有 CRUD 和元数据查询，因此容易实现，也容易验证。但当目录树有几百万个对象时，应用反复扫描全树是在浪费系统已经知道的信息：文件变化是由系统调用引发的，内核本来就在变化发生的瞬间看到了它。

2.2 inotify：把变化变成文件描述符上的事件

Linux 的 inotify 把文件变化监控设计成文件描述符模型。应用调用 `inotify_init` 或 `inotify_init1` 创建一个实例，再用 `inotify_add_watch` 把目录或文件加入监控集合。之后，应用可以像读普通文件描述符一样读取事件，也可以把这个描述符加入 `select/poll/epoll`。



... +430 lines (ctrl+o to expand)

● Read 1 file (ctrl+o to expand)

● inotify(7) 解读

inotify 是 Linux 内核提供的文件系统事件监控机制，本质上是一个内核事件通知框架。

核心接口（三个系统调用）

调用	作用
<code>inotify_init()</code> / <code>inotify_init1()</code>	创建一个 inotify 实例，返回一个文件描述符
<code>inotify_add_watch()</code>	注册对某个文件/目录的监控，返回监控描述符 (wd)
<code>inotify_rm_watch()</code>	移除监控

事件数据结构

```
struct inotify_event {
    int wd; // 监控描述符，标识是哪个 watch 产生了事件
    uint32_t mask; // 事件类型掩码
```

图 3: inotify API：创建实例、注册 watch、移除 watch，并从返回的文件描述符读取 `inotify_event`。³

```
1 int fd = inotify_init1(IN_NONBLOCK | IN_CLOEXEC);
2 int wd = inotify_add_watch(fd, path,
```

³视频画面时间区间：00:09:55-00:10:00；附近讲解：inotify 是 Linux 内核提供的事件通知框架，核心接口包括 `inotify_init`、`inotify_add_watch` 和 `inotify_rm_watch`。

```

3     IN_CREATE | IN_DELETE | IN_MODIFY | IN_MOVED_FROM | IN_MOVED_TO);
4
5 for (;;) {
6     /* read(fd, buffer, sizeof buffer) returns one or more inotify_event
7        */
8     /* fd can be managed together with sockets, pipes, timers by epoll */
9 }

```

Listing 1: inotify 的典型控制流

把监控对象抽象成文件描述符有两个重要收益。第一，它融入 UNIX 的 I/O 复用模型，应用不需要为文件监控单独引入一种等待机制。第二，它把文件系统事件变成数据流：事件结构体包含 watch descriptor、mask、cookie、name 等信息，应用可以按队列消费。

2.3 watchdog 示例：事件比想象更细

课堂用 Python watchdog 封装演示了递归监控。创建一个文件 a.txt 时，输出不只是 created，还会出现父目录 modified、文件 opened、文件 modified、文件 closed 等事件。创建深层路径时，变化不会无限向所有祖先目录传播；直接父目录的目录项改变，父目录 mtime 会变化，但更高层目录不一定变化。

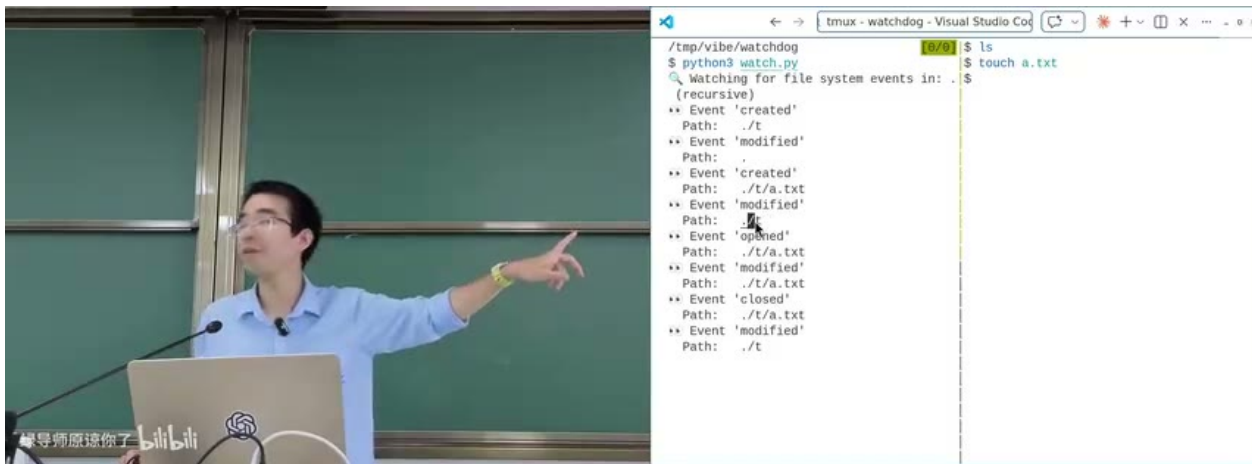


图 4: Python watchdog 递归监控示例：一次 touch a.txt 会产生创建、目录修改、打开、修改和关闭等多条事件。⁴

目录也是文件

目录保存的是名字到对象的映射。创建或删除子项时，改变的是父目录这个对象的内容，因此父目录的元数据会改变。这个细节解释了为什么文件监控常常会看到比“我只改了一个文件”更多的事件。

⁴视频画面时间区间：00:12:25–00:12:30；附近讲解：演示 touch 文件时 watcher 输出多个事件，并强调目录本身也是文件系统对象，目录项变化会修改父目录。

2.4 本章小结

监控需求可以用轮询实现，但 `inotify` 让内核在变化发生时把事件交给应用。它的设计延续 UNIX 文件描述符传统，因此可以和网络、管道、定时器一起进入统一事件循环。理解监控 API 的真正价值，不是记住函数名，而是看到“系统已经知道的状态变化”如何变成应用可消费的数据流。

3 从文件监控到操作系统状态监控

3.1 `trace` 是状态机执行的信息投影

课堂从 `inotify` 自然推进到更一般的 `trace`。文件系统监控只观察目录树变化；但操作系统本身也是状态机，内核函数调用、结构体更新、块设备 I/O、系统调用、库函数调用，本质上也都是状态变化。我们已经见过 `strace`、`ltrace`、`inotify` 和块设备 I/O 追踪，它们都是把某一类状态变化投影出来。

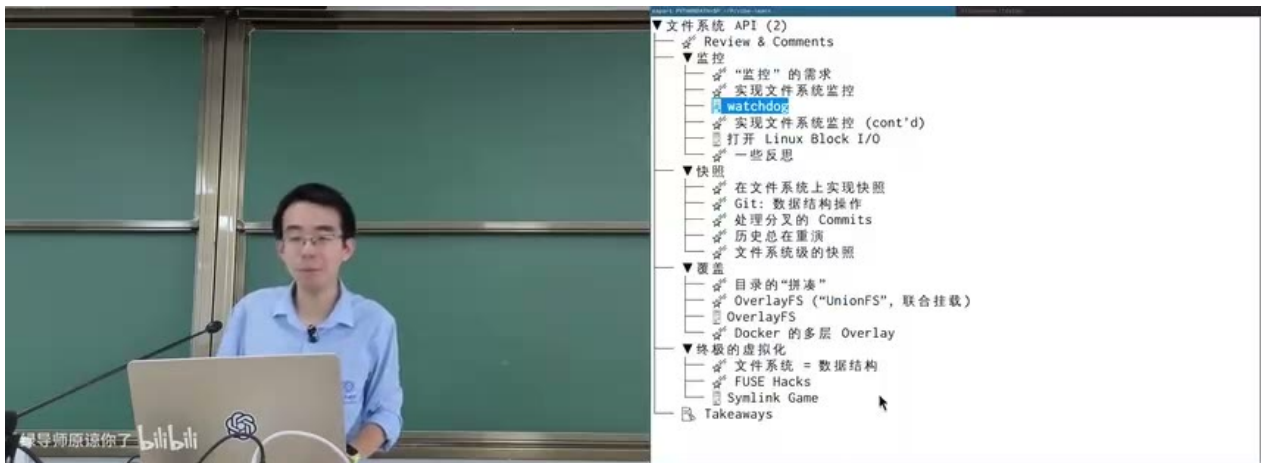


图 5: 讲义大纲从文件系统监控推进到更一般的系统状态监控，并把后续主题组织为快照、覆盖和终极虚拟化。⁵

3.2 eBPF: 把受限代码放到内核 hook point

如果我们希望在内核关键路径上提取信息，一个自然想法是：把一小段受限程序放到内核指定 hook point 上运行。每次系统经过这个点，程序就能读取上下文、统计、过滤、上报。eBPF 正是这种机制的现代形态。

⁵ 视频画面时间区间：00:13:25–00:13:30；附近讲解：在 `watchdog` 演示之后，课程回到大纲，准备从“监控文件变化”推广到“监控操作系统运行时状态变化”。

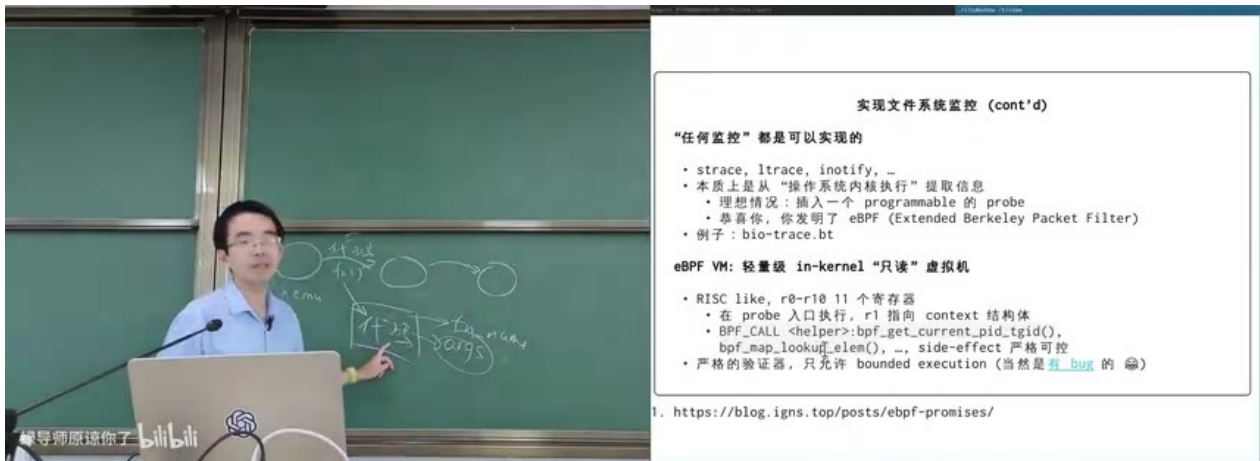


图 6: eBPF 机制: 轻量级 in-kernel 只读虚拟机, 受验证器约束, 可挂到 tracepoint/kprobe 等位置提取系统信息。⁶

eBPF 程序通常经历几步:

- 用 C 或专用前端写一段受限程序;
- 编译为 eBPF 字节码;
- 通过 bpf 系统调用加载到内核;
- verifier 检查是否越界、是否可能无限循环、是否非法访问内核内存;
- JIT 编译为本机指令, 挂载到 tracepoint、kprobe、socket filter 等 hook point;
- 通过 ring buffer、perf buffer 或 BPF map 把结果交回用户态。

eBPF 强大但不等于任意内核代码

eBPF 的力量来自“靠近内核执行路径”, 安全来自“严格受限”。不能随意访问内核内存, 不能写无限循环, 复杂状态需要借助 BPF map。课程还提到真实生产环境中 verifier/JIT 也可能有 bug, 因此 eBPF 是强工具, 不是免验证的魔法。

3.3 Specification is all you need

课程在这一段引出一个很有时代感的观点: 今天的软件开发者可以先写正确但低效的 specification, 再让模型帮助寻找更高效的系统机制。例如, 先用 stat + diff 描述“我想知道目录树哪些文件变了”, 再追问如何借助 Linux 机制高效实现。模型会收敛到 inotify、fanotify, 甚至在更一般的追踪需求下想到 eBPF。

⁶视频画面时间区间: 00:20:50–00:20:55; 附近讲解: eBPF 程序像一个小虚拟机, 有 RISC-like 指令、寄存器、helper call、map, 且必须通过 verifier 的 bounded execution 检查。

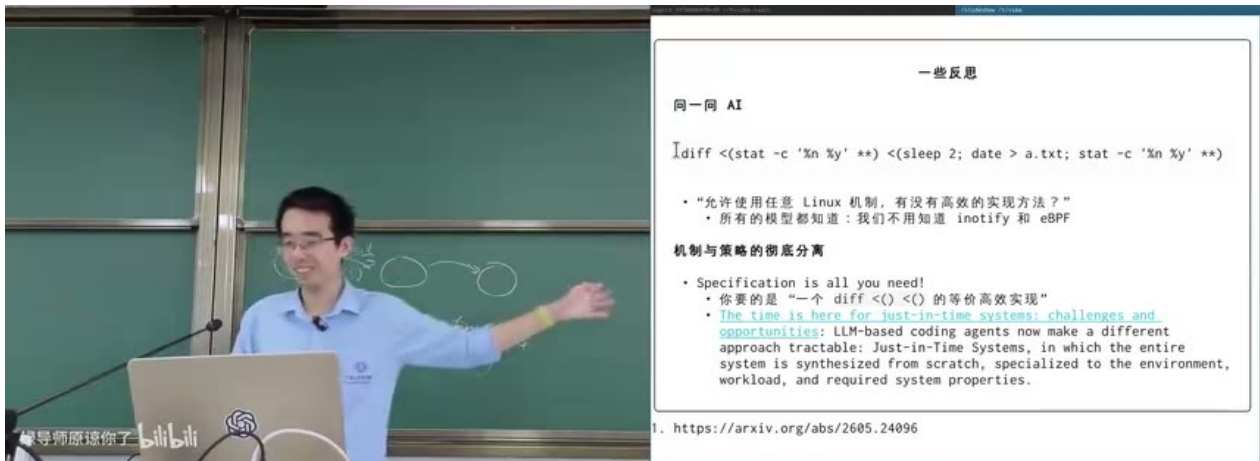


图 7: 用低效但正确的轮询 specification 追问高效机制: 模型可从需求描述收敛到 inotify/fanotify/eBPF 等系统能力。⁷

能力结构的变化

未来更重要的能力不是背下每个库的用法, 而是清楚知道系统底层可以提供哪些基本能力, 并能把需求写成可验证的 specification。性能实现可以借助模型和系统机制, 但正确性、边界条件、质量保障仍然需要工程判断。

3.4 本章小结

从 inotify 到 eBPF, 思路从“监控文件系统事件”扩展到“从操作系统状态机执行中提取信息”。这也解释了为什么操作系统知识在 AI 时代并没有贬值: 你越理解系统能做什么, 就越能提出好 specification, 让工具替你找到高效实现路径。

4 快照: 持久化数据结构与 Git

4.1 append-only + random read

快照来自持久化数据结构。若数据结构永不原地覆盖, 而是每次修改只写新节点, 并让新根指向新路径、旧根仍指向旧路径, 那么所有历史版本都可以保留。对于树结构, 插入一个节点时只需要复制从根到修改点的一条路径, 共享未改变的子树。

⁷ 视频画面时间区间: 00:31:10–00:31:15; 附近讲解: Specification is all you need, 先把想要的行为描述清楚, 再借助模型把它映射到已有操作系统机制。



图 8: 课程把文件系统快照归结为 random read + append-only write: Git 的 blob/tree/commit 对象就是持久化目录树的具体例子。⁸

这个模型可以被概括为:

$$\text{snapshot} = \text{name}(\text{root pointer at time } t)$$

符号说明:

- root pointer: 指向某一版本目录树根节点的引用;
- t : 创建快照的逻辑时间;
- name: 给这份根引用起的名字, 例如 Git ref、Btrfs subvolume snapshot 名称;
- 快照本身不需要复制所有数据, 复制发生在后续写入触碰到共享节点时。

4.2 Git 对象: blob、tree、commit

Git 是课堂中最重要的例子。一个干净仓库里, 所有历史都保存在 `.git` 下。文件内容是 blob; 目录树是 tree; 一次提交是 commit, commit 里记录 tree、parent、author、message 等信息。分支不过是 refs/heads/branch 里的一个哈希; HEAD 是指针的指针; stash 也可以理解成带特殊 parent 结构的 commit。

⁸视频画面时间区间: 00:32:25–00:32:30; 附近讲解: 切入快照主题, 指出 Git 是大家熟悉的 persistent data structure, 三类对象实际存放在 `.git/objects` 中。

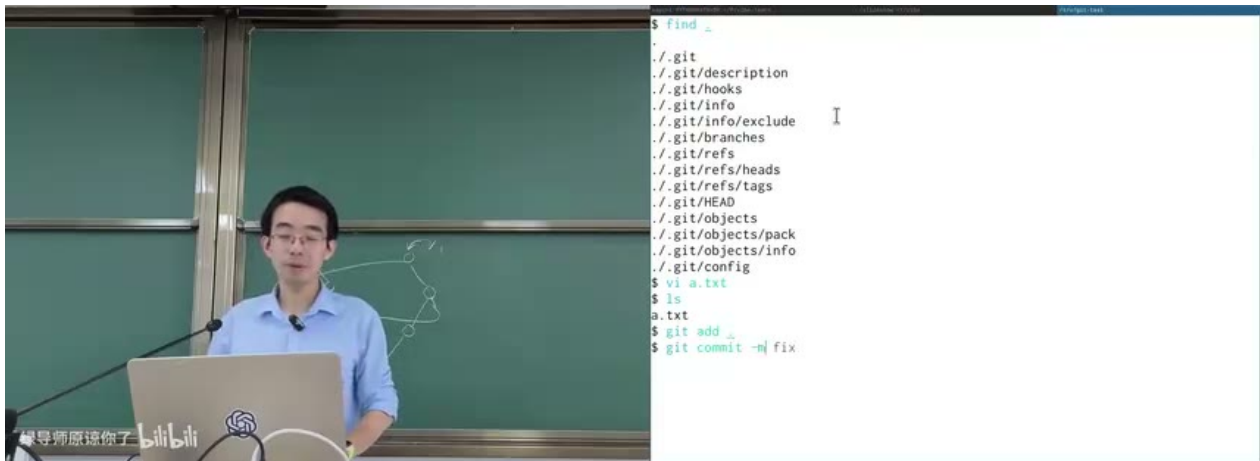


图 9: Git 仓库演示: .git 中保存对象池、引用和 HEAD; git add/git commit 本质上是在持久化对象图中追加节点并移动引用。⁹

```

1 # blob: file content
2 # tree: directory entries, mode, filename, object hash
3 # commit: tree hash + parent commit hash(es) + metadata + message
4
5 cat .git/HEAD
6 cat .git/refs/heads/main
7 find .git/objects -type f

```

Listing 2: Git 对象模型的最小心智模型

这个设计非常 UNIX: 一切都是文件, 指针是文本, 内容寻址由哈希完成, 对象追加后基本不变。正因如此, Git 可以免费拥有许多高级能力:

- 分支: 新增一个 ref 指向已有 commit;
- checkout: 把工作区恢复为某个 tree;
- reflog: 记录引用曾经指向过哪些 commit, 便于找回历史;
- stash: 把未完成工作保存成 commit-like 对象;
- worktree: 多个工作目录共享同一个对象池, 只持有不同 HEAD 和工作区。

rebase 的风险

rebase 可以理解为把一串 diff 逐个 cherry-pick 到新的基底上, 制造“我是在别人最新提交之后继续开发”的线性历史假象。如果分支分叉时间很长, 冲突会逐个出现; 即便自动成功, 也可能隐藏逻辑冲突。课堂建议: 遇到复杂冲突时, merge 往往比强行 rebase 更稳。

⁹视频画面时间区间: 00:34:00–00:34:05; 附近讲解: 创建空 Git 仓库, 查看 .git 目录, 提交 a.txt 后观察 objects 和 refs 的变化。

4.3 Btrfs: 把整个文件系统做成持久化数据结构

如果 Git 能把项目目录做成持久化数据结构，文件系统本身为什么不能？Btrfs 的思路就是把文件系统元数据和数据组织成 copy-on-write 的 B-tree。它不是把对象放进已有文件系统的 `.git/objects`，而是在块设备上从零实现一个持久化目录树。创建快照时，本质上也是保存一个根指针；之后写入触发 copy-on-write，旧快照仍然看到旧块。

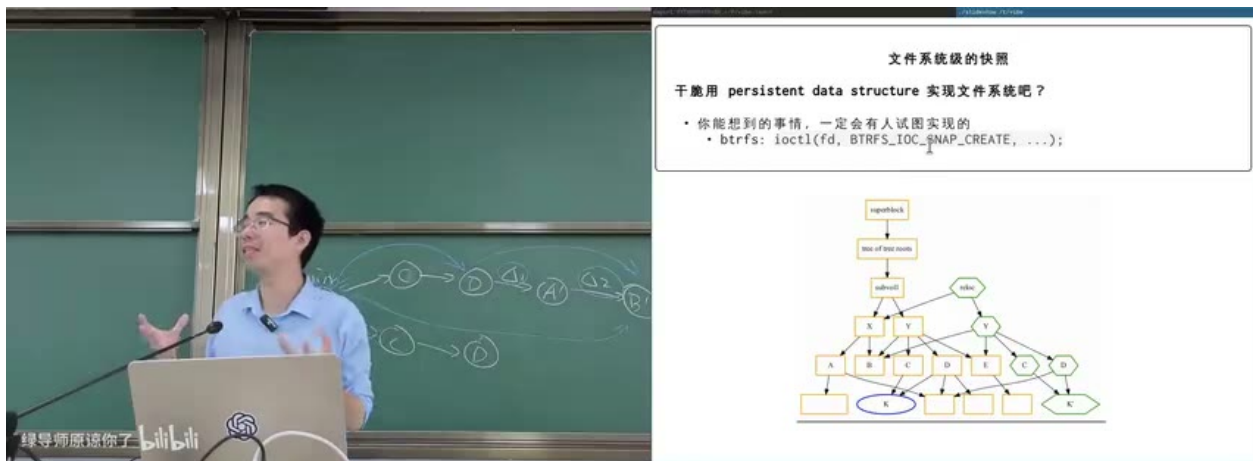


图 10: Btrfs 快照：文件系统自身作为 copy-on-write 持久化数据结构，创建 snapshot 近似为给当前根指针起名。¹⁰

快照不是备份

快照保存的是某一时刻的文件系统视图，通常依赖底层共享块。如果底层设备整体损坏，快照也可能一起损坏。备份要求把数据复制到独立故障域。快照适合快速回滚、版本保留、实验隔离；备份负责灾难恢复。

4.4 本章小结

快照的核心不是“复制一份目录”，而是持久化数据结构中的根引用管理。Git 在普通文件系统上实现这个思想；Btrfs 在块设备层实现这个思想。理解 blob/tree/commit、refs、HEAD、worktree 之后，许多 Git 高级命令都不再神秘：它们只是对象图上的引用更新、diff replay 或新节点追加。

5 OverlayFS 与 Docker：目录树的叠加语义

5.1 UnionFS/OverlayFS 的新语义

监控和快照仍可看作 CRUD 的高效或持久化变体：监控可以用轮询模拟，快照可以用整树复制模拟。Overlay 更进一步，它改变了目录树本身的语义：把一个或多个 lower 目录

¹⁰ 视频画面时间区间：01:00:00–01:00:05；附近讲解：从 Git 推广到 Btrfs，使用 `BTRFS_IOC_SNAP_CREATE` 等接口创建文件系统级快照。

和一个 upper 目录叠成一个 merged 目录。查询时先看 upper，再看 lower；修改 lower 中的文件时，写入 upper；删除 lower 文件时，在 upper 中放 whiteout 标记。

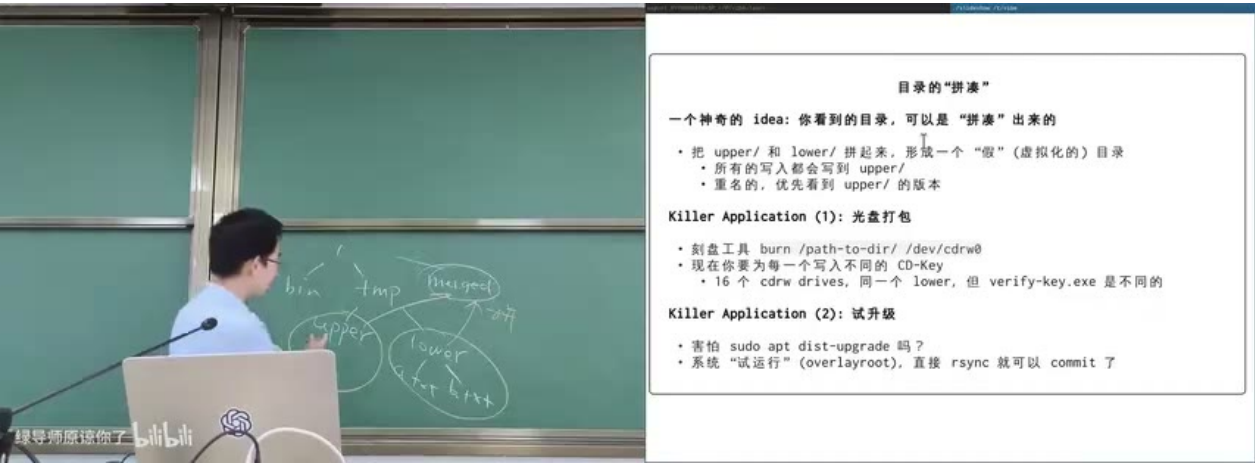


图 11: 目录的“拼凑”: upper/lower 合成 merged 视图, 可用于光盘打包、试升级等共享底层、隔离修改的场景。¹¹

OverlayFS 典型挂载形式如下:

```
1 mount -t overlay overlay \
2   -o lowerdir=/lower1:/lower2,upperdir=/upper,workdir=/work \
3   /merged
```

Listing 3: OverlayFS 的典型 mount 参数

其中 workdir 是内核需要的临时目录; upperdir 只有一个; lowerdir 可以有多个。合成目录对用户看起来像普通目录, 但背后每次 lookup、read、write、rename、unlink 都要按 overlay 规则解释。

Overlay 的本质

Overlay 不是“把目录复制到一起”, 而是创建一个新的解释层。merged 目录不是独立副本, 而是一种视图: 读操作从 upper/lower 解析, 写操作落到 upper, lower 层可以被多个视图共享。

5.2 应用：试升级、考试环境、网吧管理

课程列举了很生活化的场景。若系统根目录作为 lower, 另一个空目录作为 upper, 那么一次高风险升级的所有写入都会进入 upper。升级成功时, 可以把 upper 的改动提交回真实系统; 失败时, 丢弃 upper, 就回到旧系统。考试机和网吧管理也类似: 底层系统共享, 用户会话写入 upper, 下机或换场时更换 upper。

¹¹ 视频画面时间区间: 01:02:25–01:02:30; 附近讲解: UnionFS/OverlayFS 把 upper 和 lower 拼成虚拟目录, 访问时优先 upper, lower 保持不变。

whiteout 是“删除”的影子

如果 lower 中有 `lower.txt`，用户在 merged 中删除它，不能真的改 lower。OverlayFS 会在 upper 中创建 whiteout，表示“在这个视图里它被删除了”。这解释了为什么 overlay 目录中会出现一些看似奇怪的特殊文件或字符设备。

5.3 Docker layer：每个 RUN 都是一层

Docker 的镜像层把 OverlayFS 思想工业化。Dockerfile 从基础镜像开始，每个 RUN 在上一层基础上创建一个可写 upper，命令结束后 upper 固化为新的只读 layer。下一条命令再在这些 lower 之上创建新的 upper。

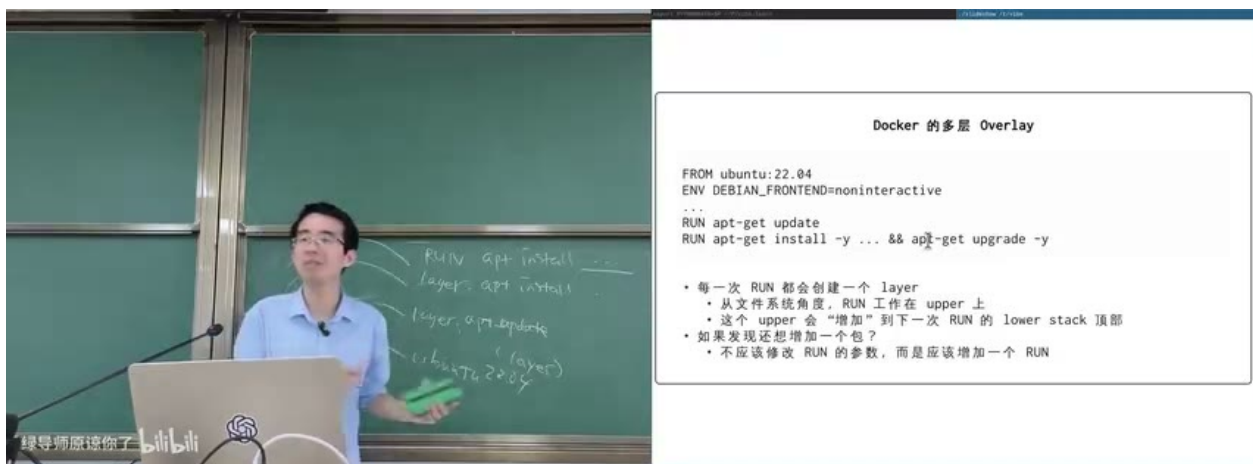


图 12: Docker 的多层 Overlay：每条 RUN 生成一层，修改早期命令会导致其后的层重新 replay。¹²

这解释了几个 Dockerfile 经验规则：

- 修改靠前层会使后续缓存失效，因为后续层依赖前一层的文件系统快照；
- 若只是漏装一个包，通常在后面新增 RUN 或重排命令比修改很早的基础层更便宜；
- 多个容器可共享相同只读镜像层，只为每个运行中的容器分配自己的可写层；
- 容器不是把整个 rootfs 复制一份，而是共享 lower，隔离 upper。

5.4 本章小结

OverlayFS 引入的是目录树合成语义。它超出了 CRUD，却仍然通过 mount 接入统一目录树。Docker 镜像、试升级、考试机隔离等场景都依赖同一个抽象：共享不可变底层，把变化限制在可丢弃、可提交、可替换的 upper 层。

¹²视频画面时间区间：01:14:55–01:15:00；附近讲解：Dockerfile 从 `ubuntu:22.04` 开始，`apt-get update`、`apt-get install` 等命令逐层产生 layer。

6 FUSE: 把任意数据结构伪装成文件系统

6.1 为什么不能自己实现文件系统

如果文件系统是数据结构，那么最终问题就是：能不能直接写一个程序实现这个数据结构，并让操作系统把它当作文件系统？内核中的 `/proc` 已经说明答案是肯定的：某些目录项并不对应磁盘上的普通文件，而是由内核代码即时生成。

FUSE，即 Filesystem in Userspace，把这个能力交给用户态程序。内核接收 VFS 层的 `lookup`、`read`、`write`、`readdir`、`getattr` 等请求，再转发给用户态 FUSE daemon；用户态程序实现回调，返回目录项、文件内容、属性或错误码。

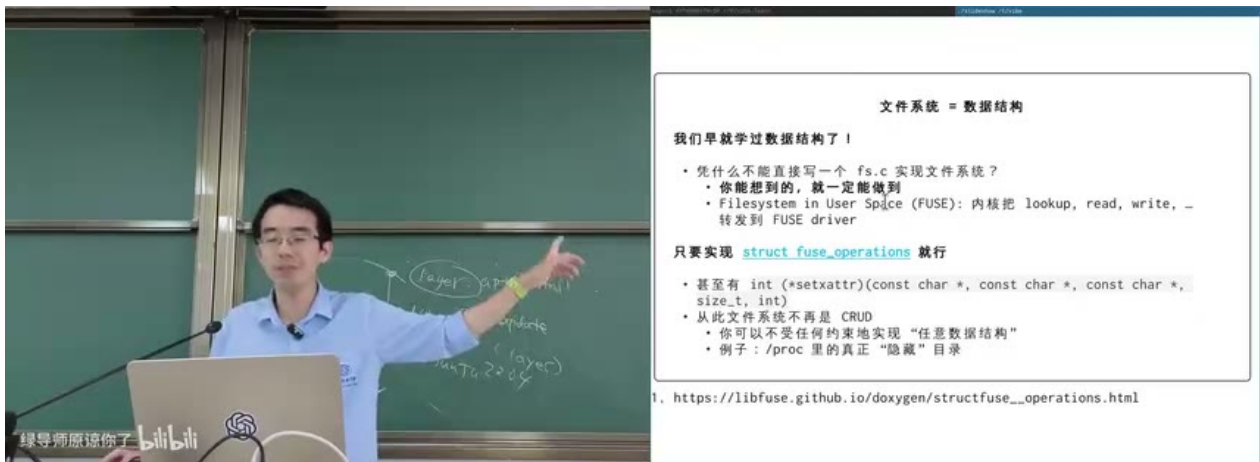


图 13: FUSE 的核心接口：实现 `struct fuse_operations`，让用户态程序决定文件系统的 `getattr/read/write/readdir` 等行为。¹³

```
1 static struct fuse_operations ops = {
2     .getattr = my_getattr,
3     .readdir = my_readdir,
4     .open    = my_open,
5     .read    = my_read,
6     .write   = my_write,
7 };
```

Listing 4: FUSE operations 的概念结构

6.2 GGFS: 文件系统驱动 Galgame

课堂演示了一个 GGFS，即把 galgame 状态机挂载成文件系统。挂载后，`/tmp/ggfs` 下出现虚拟文件；读文件可以返回程序生成的内容；向某个文件写入数据会触发内部状态变化，目录内容随之改变。这时“文件”已经不是磁盘字节，而是程序状态机的一组接口。

¹³视频画面时间区间：01:17:15–01:17:20；附近讲解：内核把文件系统操作转发给 FUSE driver，用户只需实现 `struct fuse_operations`，从此文件系统不再只是 CRUD。

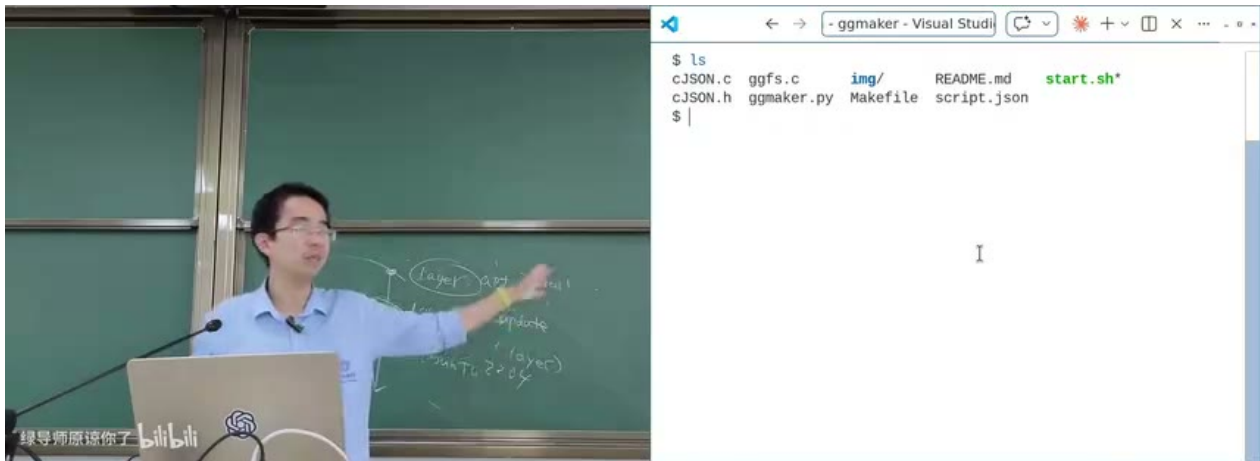


图 14: GGFS demo 目录: ggfs.c、脚本、JSON 和资源文件共同实现一个由文件操作驱动的虚拟文件系统。¹⁴

6.3 FUSE hacks: 把非文件变成文件

FUSE 的想象空间很大:

- sshfs: 把远程 SSH 目录挂成本地目录;
- aifs: 把操作转给 AI agent, 让模型参与目录内容生成;
- dbfs: 用数据库保存文件系统状态, 把文件操作翻译为 SQL;
- ffs: 把 JSON/key-value 等非文件结构映射成目录和文件;
- ggfs: 用文件读写驱动游戏或应用状态机;
- 隐藏目录: readdir 不列出, 但知道名字仍可 cd, 展示文件系统语义由实现决定。

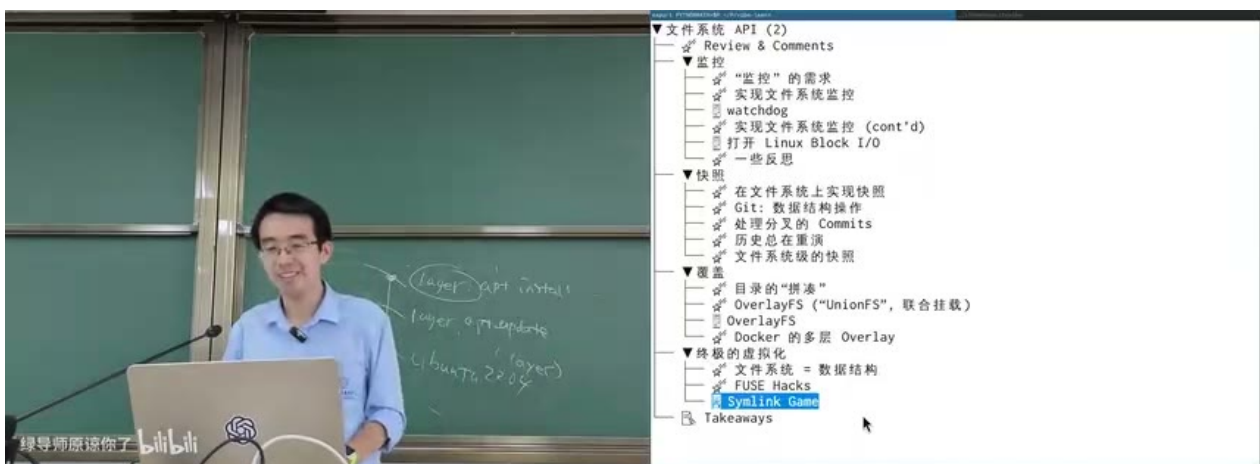


图 15: 课程大纲回到“终极的虚拟化”: 文件系统本身可以由数据结构或程序实现, 普通路径名成为访问这个实现的接口。¹⁵

¹⁴ 视频画面时间区间: 01:19:05–01:19:10; 附近讲解: 进入 ggmaker 目录, 准备构建并挂载 galgamefs, 随后通过读写虚拟文件推进应用状态。

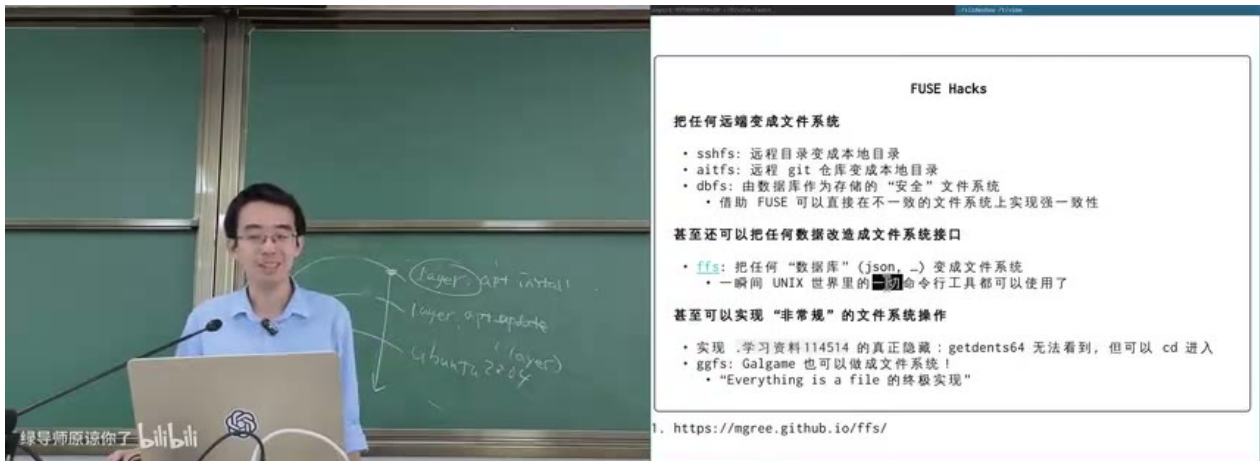


图 16: FUSE hacks 总结: sshfs、aiffs、dbfs、ffs 和 ggfs 都是在把非传统文件对象重新包装成文件系统。¹⁶

Everything is a file 的终极版本

传统 UNIX 说 everything is a file, 强调设备、管道、终端等都可通过文件接口访问。FUSE 把这句话推进一步: 只要能够实现一组回调, 任意数据结构、服务、数据库、远程资源或交互状态机, 都可以被包装成文件系统。

6.4 本章小结

FUSE 是文件系统虚拟化的用户态入口。它把 VFS 请求转发给用户程序, 让程序定义目录项、文件内容、属性和副作用。于是文件系统不再必须来自磁盘, 也不必遵循普通 CRUD 直觉; 它可以是网络代理、数据库视图、AI 接口, 甚至是游戏状态机。

7 总结与延伸

7.1 课程结论

这一讲从很朴素的文件变化监控出发, 逐步走到文件系统作为可编程抽象的全景:

- inotify 把文件系统变化转换为文件描述符上的事件流;
- eBPF 把受限程序放进内核 hook point, 把操作系统状态变化变成可观测数据;
- Git 和 Btrfs 展示了 append-only、copy-on-write、root pointer 命名如何带来快照;
- OverlayFS 把多个目录层合成一个视图, Docker 镜像层和容器写层建立在这个思想上;

¹⁵ 视频画面时间区间: 01:20:20–01:20:25; 附近讲解: 大纲高亮终极虚拟化, 包含文件系统作为数据结构、FUSE hacks 和 symbolic game。

¹⁶ 视频画面时间区间: 01:22:00–01:22:05; 附近讲解: 总结 FUSE 可把远程目录、AI agent、数据库、JSON 或游戏状态机变成统一文件系统接口。

- FUSE 允许用户态代码实现文件系统，把任意数据结构包装进 UNIX 路径空间。

7.2 概念压缩

可以用一张表把本讲的机制压缩起来：

机制	数据结构观点	API/实现入口
监控	变化日志/事件队列	<code>inotify</code> , <code>fanotify</code> , <code>watcher</code>
系统追踪	状态机执行投影	<code>strace</code> , <code>tracepoint</code> , <code>eBPF</code>
快照	持久化根指针	Git refs, Btrfs snapshot ioctl
覆盖	多层目录树视图	<code>mount -t overlay</code>
容器层	append-only rootfs history	Docker layer + writable upper
FUSE	用户态实现 ADT	<code>struct fuse_operations</code>

本讲的一句话

文件系统 API 的高级部分，实质是在问：除了对目录树做增删改查，我们还能不能观察它、命名它的历史、叠加它的视图、替换它的实现？答案都是可以，而且这些机制已经构成现代开发工具链、容器、调试和系统扩展的基础。

7.3 面向 AI 时代的延伸

课程中反复强调的不是“让 AI 替代操作系统知识”，而是相反：当模型能帮你查接口、写代码、解释工具时，真正稀缺的是能提出好 specification 的人。你要能说清楚：

- 需要观察的是文件变化、系统调用、块 I/O，还是任意内核 hook point；
- 需要的是历史版本、可回滚试验层，还是多容器共享镜像层；
- 需要的是普通文件系统语义，还是一个自定义数据结构的文件系统投影；
- 正确性边界在哪里，性能瓶颈在哪里，失败后如何恢复。

当这些问题说清楚后，模型可以帮助你从低效原型走向系统机制：从 `stat + diff` 到 `inotify`，从手写日志到 `eBPF`，从复制目录到 Btrfs snapshot，从拷贝 rootfs 到 OverlayFS，从专用 RPC 到 FUSE 文件系统。

7.4 本章小结

文件系统是操作系统里最有想象力的统一抽象之一。它从磁盘数据结构出发，却能承载事件流、版本历史、容器镜像、远程资源和用户态程序。把它看作抽象数据类型，许多看似神奇的机制就变成了熟悉的数据结构操作：追加、共享、命名、合成、解释和回调。